东舟API接口文档

东舟自动化测试平台, 通过远程调用框架(Thrift-RPC), 对外提供跨语言的调用, 支持 C++, Java, Python, PHP, Ruby, Erlang, Perl, Haskell, C#, Cocoa, JavaScript, Node.js, Smalltalk, OCaml and Delphi等常用编程语言.

API分为两大类

1. 常驻API, 软件启动之后就可以直接调用, 通常用来获取工位状态, 查询工位信息等
2. 远程模式API, 用于驱动测试动作, 执行测试用例和任务, 获取测试结果等

切换到远程模式的方法有两种

1. 通过系统菜单切换, 系统菜单工具->远程控制
2. 通过常驻API中的SetModel("remote") 切换

**部分API接口可能会抛出异常, 异常定义见结构体和枚举说明

**更多Thrift信息, 请参考 https://thrift.apache.org

Thrift通讯方式:

```
地址: 本机IP地址
端口: 默认 9300
协议: socket
```

结构体和枚举说明

结构体

```
1.异常类型
//操作无效的异常类
exception  InvalidOperation
{
    string Operation, //操作名称, 对应接口名称
    string Messgae,   //异常消息
}
```

常驻API

1 获取客户端名称, 用于区分客户端的类型.

```
返回值:
//客户端名称（例: HMIRobot, TouchRobot 等, 具体请查阅硬件信息）
string GetName();
```

2 获取客户端的唯一编号

```
返回值：
//客户端sn号（例：103D1C481071）
string GetSN();
```

3 获取客户端版本

```
返回值：
//系统版本号（例：2.2023.218.1127）
string GetVer();
```

4 获取软件的安装目录（绝对路径）

```
返回值：
//例如D:\\DZ_MachineHand\\V2.2022.1013.892_RELEASE\\
string GetRootDirectory();
```

5 获取客户端状态，用来统计使用率

```
**执行任务属于使用状态
**编辑用例，分析报告，处理数据等，鼠标键盘5分钟内有变化，属于使用状态.
返回值：
//true 使用状态，false 空闲状态
bool GetStatus();
```

6 切换远程模式

```
请求参数：
string model 模式名称：remote（远程模式），local（本地模式）
返回值：
//无
void SetModel(string model);
```

7 获取任务执行状态

```
请求参数：
//int taskId 任务的Id
返回值：
//任务的状态
**Running 表示正在执行中
**Finish 表示正常执行完
**Error 和其他返回值，表示出错了
异常信息：
1.设备急停
2.其他异常
string GetTaskState(int taskId);
```

8 获取任务执行结果

请求参数：
//int taskId 任务的Id
返回值：
//字符串,用json表示执行结果相关信息
异常信息：
1.正在执行测试任务中，无法执行当前动作
2.任务不存在
3.其他异常
```
string GetTaskResult(int taskId);
```

可用 CustomLogs 如下：
**图像识别：时间|步骤名称|执行结果|屏幕图像|图像模板
例如：
"2021-10-15 15:35:55.230|识别导航图标|成功|c:\\a.bmp|d:\\b.bmp"

**音频识别：时间|步骤名称|执行结果|音频模板路径|录音文件路径|声音幅值
例如：
"2021-10-15 15:35:55.230|录制音频|成功|c:\\a.wav|c:\\0.wav|123"

**文本识别：时间|步骤名称|执行结果|待验证文本|识别到的文本
例如：
"2021-10-15 15:35:55.230|识别导航文字|成功|导航|开始导航"

**语义识别：时间|步骤名称|执行结果|待验证内容|识别到的内容|音频路径
例如：
"2021-10-15 15:35:55.230|识别车机语音|成功|北京天气|今天北京天气晴…"

json格式如下：
```
{
    "TestExecutionLogs":[
        {
            "Id": 1,                                    // 日志 id
            "TaskId": 2,                                // 任务 id
            "SetId": 3,                                 // 用例集 id
            "CaseId": 4,                                // 用例 id
            "CaseName": "用例名称",                    // 用例名称
            "StartTime": "2023-01-10T16:21:43.695", // 用例执行开始时间
            "FinishTime": "2023-01-10T16:21:53.157",// 用例执行结束时间

            "ExeResult": true      // 用例执行结果 成功：true ,  失败：false
            "ExeResultNew": 1,     // 用例执行结果 0：失败 ,  1：成功 ,  2：错误
            "ExeTimeSapn": 9462, // 用例执行耗时（单位：ms）
            "CustomLogs":null,
            //流畅度测试结果举例
            "ResultDetail": {
                "TotalCaton": 0,           //卡顿总帧数
                "MaxContinuousCaton": 0,  //最大连续卡顿帧数
                "RatioCaton": "60%",       //卡顿比率
                "AverageFrame": 0,         //平均帧率
                "FilePath": "20210819155042799" //图像文件夹名称
            },
            "MarkTimeSpan": 0,            // (响应时间，默认=0)
            "WorkFolder": "D:\logs\Robot\taskLogs\1\5\20210819155042702",
```

```
            //可靠性测试结果举例
            "ResultDetail":{
                "TouchPointCheck": {
                    "Name": "TouchPointCheck",
                    //结果存储类型，0：存储于数据库和文件 1：仅数据库 2：仅文件
                    "StorageType": 2,
                    "FilePath": "D:\\20240111152717382\\errorOutput.log",
                    "Data": "2024-01-11 15:27:18.575/2024-01-11 15:27:22.953/矩阵滑动测试/ADB/全程无报点\r\n2024-01-11 15:27:22.955/2024-01-11 15:27:31.665/矩阵点击测试/ADB/出现漏报点,理论按下数量：5，实际按下数量：4，实际按下报点列表：[00:00:00 367,113];[00:00:00 366,362];[00:00:00 116,111];[00:00:00 114,361];"
                }
            },

        }
    ],
    "ret": true,          //接口执行结果 成功： true ， 失败： false
    "error": null         //结果产生错误信息
}
```

9 获取所有任务

返回值：

```
//任务集合对象
string GetAllTasks()
```

远程模式API

1. 基本信息

1.1 获取终端工位信息

返回值：

```
//字符串,用json表示的启用的终端工位信息
json格式如下：
[
    {
        "DeviceNo": 1,                                      //工位编号
        "Name": "工位名称",                                  //工位名称
        "RectByCamera":{X=20, Y=20, Width=100,Height=50}, //屏幕范围，表示x，y，宽，高
        "Keys": ["Home","Back","Menu"]                      //工位按键
    }
]
string GetDevices();
```

1.2 检查硬件状态

```
**注意：检查硬件状态一般用在执行任务之前，查看硬件是否出现了异常.
***切勿在执行过程中使用此函数
**常见硬件类型 相机Camera，机械手Robot，声卡SoundCard，程控电源开关PowerSwitch，USB开关USBSwitch，霍尔感应器HollSwitch
**具体的硬件类型和数量，请查阅硬件信息
```

返回值：
//字符串，用json表示的各个硬件的状态信息
json格式如下：
```
[
    {
        "Type":"Camera",  //硬件类型
        "Id":"xxxxxx",    //硬件id
        "Active":true,    //是否正常
        "ErrMsg":""       //硬件不正常时的错误消息
    }
]
```
```
string GetHWStatus();
```

2.机械手控制

约定：
1.）机械手控制API中的工位号deviceNo，可以从GetDevices接口的返回值中读取DeviceNo
2.）机械手控制API中的坐标，除非特殊说明， 都是以工位的图像尺寸为单位，范围可以从GetDevices接口的返回值中读取RectByCamera

2.1 机械手回零（正常速度回零）

请求参数：
//int deviceNo   工位号
//string handname 机械手名称:主手、副手
异常信息：
1.正在执行测试任务中，无法执行当前动作
2.其他异常
```
void GoHome(int deviceNo, string handname);
```

2.2 机械手回零（快速回零）

请求参数：
//int deviceNo   工位号
//string handname 机械手名称:主手、副手
异常信息：
1.正在执行测试任务中，无法执行当前动作
2.其他异常
```
void FastGoHome(int deviceNo, string handname);
```

2.3 点击屏幕

请求参数：
```
//int deviceNo  工位号
//double x      X坐标
//double y      Y坐标
//int clickType 点击类型,0点击, 1双击, 2长按
//long timeout  长按时间,当clickType=2时有效
```
异常信息：
1.正在执行测试任务中，无法执行当前动作
2.工位号无效(未启用或不存在)
3.坐标超出屏幕范围
4.点击类型无效
```
void Click(int deviceNo, double x, double y, int clickType, long timeout);
```

2.4 滑动屏幕

请求参数：
```
//int deviceNo 工位号
//double x1    X1起点坐标
//double y1    y1起点坐标
//double x2    X2终点坐标
//double y2    y2终点坐标
//int speed    滑动速度：必须大于0
```
异常信息：
1.正在执行测试任务中，无法执行当前动作
2.工位号无效(未启用或不存在)
3.坐标超出屏幕范围
```
void Swipe(int deviceNo, double x1, double y1, double x2, double y2, int speed);
```

2.5 连续滑动

请求参数：
```
//int deviceNo        工位号
//double[] pointArgs double数组; x1,y2,x2,y2,x3,y3....xn,yn 数组的长度是偶数，如果是奇数，最后一个数代表每段等待时间
//int speed           滑动速度：必须大于0
```
异常信息：
1.正在执行测试任务中，无法执行当前动作
2.工位号无效(未启用或不存在)
3.坐标超出屏幕范围
```
void ContinueSwipe(int deviceNo, double[] pointArgs, int speed);
```

2.6 曲线滑动 （仅支持机械手类型客户端）

```
**注意：曲线滑动只能沿着x轴或者y轴滑动， 当沿着x轴滑动时y1=y2， 当沿着y轴滑动时x1=x2
请求参数：
//int deviceNo 工位号
//double x1    X1起点坐标
//double y1    y1起点坐标
//double x2    X2终点坐标
//double y2    y2终点坐标
//int speed    滑动速度：必须大于0
异常信息：
1.正在执行测试任务中，无法执行当前动作
2.工位号无效(未启用或不存在)
3.坐标超出屏幕范围
void CurveSwipe(int deviceNo, double x1, double y1, double x2, double y2, int
speed);
```

2.7 点击按键

```
请求参数：
//int deviceNo   工位号
//string keyName 按键名称:可以从GetDevices接口的返回值中读取 Keys
//int clickType  点击类型:0 点击, 1 双击, 2 长按
//long timeout   长按时间:当clickType=2时有效
异常信息：
1.正在执行测试任务中，无法执行当前动作
2.工位号无效(未启用或不存在)
3.按键不存在
4.点击类型无效
void ClickKey(int deviceNo, string keyName, int clickType, long timeout);
```

2.8 连续点击

```
请求参数：
//double x             X坐标
//double y             Y坐标
//int deviceNo         工位号
//double zDelta        Z轴增量
//int times            连续点击次数
//int interval         连续点击间隔
//bool isRobotPostion 是否支持双手坐标
异常信息：
1.正在执行测试任务中，无法执行当前动作
2.工位号无效(未启用或不存在)
3.坐标超出屏幕范围
void GetContinuousClick(double x, double y, int deviceNo, double zDelta = 0, int
times = 1, int interval = 0, bool isRobotPostion = false)
```

2.9 拖拽滑屏

```
请求参数：
//int x1             X1起点坐标
//int y1             y1起点坐标
//int x2             X2终点坐标
//int y2             y2终点坐标
```

```
//int pressPauseTime 第一个点按下的等待时间
//int swipePauseTime 执行去下一个点的等待时间
//int speed          滑动速度:必须大于0
//string direction   滑动方向:"left、right、up、down"
//int deviceNo       工位号
//string stepDesc    步骤描述
//string stepImage   步骤图片
异常信息:
1.正在执行测试任务中，无法执行当前动作
2.工位号无效(未启用或不存在)
3.坐标超出屏幕范围
void GetDrag(int x1, int y1, int x2, int y2, int pressPauseTime, int
swipePauseTime, int speed, string direction, int deviceNo, string stepDesc,
string stepImage)
```

3.相机和图像操作

```
约定:
1.) 相机操作中用到的 cameraIndex, 从0开始计数, 每个相机对应的功能请咨询技术服务工程师
2.) 图像文件,一般都是32位ARGB彩色图像, 部分高速相机是灰度图, 详情请咨询技术服务工程师
```

3.1 开始采集图像

```
请求参数:
//int cameraIndex 相机索引编号
//string savePath 保存路径, 是一个文件夹名称
返回值:
//相机存图完整路径
异常信息:
1.相机索引编号无效(相机未开启或不存在)
2.图像路径不存在
3.其他异常
string StartImageGrab(int cameraIndex, string savePath);
```

3.2 结束采集图像

```
请求参数:
//int cameraIndex 相机索引编号
异常信息:
1.相机索引编号无效(相机未开启或不存在)
2.其他异常
void StopImageGrab(int cameraIndex);
```

3.3 获取相机单张图像

请求参数：
//int cameraIndex 相机索引编号
//string saveFile 文件名：需要提前创建好路径
异常信息：
1.相机索引编号无效(相机未开启或不存在)
2.文件路径异常
3.其他异常

```
void GetOneImage(int cameraIndex, string saveFile);
```

3.4 获取终端屏幕的图像

**注意：是在相机单张图像的基础上，根据工位的RectByCamera截取之后的图
请求参数：
//int deviceNo 工位号
//string saveFile 文件名：需要提前创建好路径
异常信息：
1.正在执行测试任务中，无法执行当前动作
2.工位号无效(未启用或不存在)
3.文件路径异常
4.其他异常

```
void ScreenShot(int deviceNo,string saveFile);
```

3.5 模板匹配

**注意：通过模板匹配结果，可以判断图像是否存在，结合点击动作，可以实现点击图像的操作
请求参数：
//int deviceNo 工位号
//string imageFile 模板图像：节点1/节点2/模板图名称
//double exhaustive 阈值：0.1~1
//string iconType 图标类型：普通图标=Normal;透明图标=Transparent,特征匹配=Feature
//string rectLimit x,y,width,height表示的限定区域，空字符串为不限制
返回值：
//字符串，用json表示的匹配结果信息
json格式如下：

```
{
    "BestValue": 0.75,      //最佳匹配值
    "BestPoint": "10, 20", //匹配位置 x,y
    "MatchSize": "50, 20", //匹配到的尺寸 width,height
}
```

异常信息：
1.正在执行测试任务中，无法执行当前动作
2.工位号无效(未启用或不存在)
3.模板图不存在
4.限制区域无效
5.其他异常

```
string MatchTemplate(int deviceNo, string imageFile, double exhaustive, string
iconType,string rectLimit);
```

3.6 识别文本

```
**注意：识别结果需要自己连接起来对比.
**通过识别结果，可以判断文本是否存在,结合点击动作，可以实现点击文本的操作
请求参数:
//int deviceNo      工位号
//string rectLimit 限制区域：空字符串表示全图识别
//int angle         文本角度(顺时针)：支持 0,90,180,270
返回值:
//字符串，用json表示的识别结果
json格式如下:
{
    "result" : 0,                    //0为识别成功，非0为识别异常
    "second" : 0.33770499999999998, //用时，单位s
    "textlist" :                     //根据list是否为空，可以判断是否识别到文本
    [
        {
            "location" : "40,31,54,25",    //文本内容在图像中的位置 x,y,width,height
            "score" : 0.96656703948974609,
            "text" : "86"                  //识别到的内容
        }
    ]
}
异常信息:
1.正在执行测试任务中，无法执行当前动作
2.工位号无效(未启用或不存在)
3.限制区域无效
string OCR(int deviceNo, string rectLimit, int angle);
```

3.7 设置帧率,曝光,增益

```
请求参数:
// 1:int cameraNo      相机索引编号
// 2: int frame         帧率（暂时不支持定位相机的帧率修改）
// 3: int exposureTime 曝光时间
// 4: double gain      增益值
void SetCamera(int cameraNo, int frame, int exposureTime, double gain)
```

4.用例和任务

```
任务的使用流程一般是: 执行用例(或者任务)->查询执行状态->等待执行结束->导出报告或者获取执行结果数据
执行过程中，如果需要停止任务，可以调用 StopTask
**任务执行过程中，如果又调用了相同工位的任务或者用例，会导致上一个没执行完的任务提前结束.
```

4.1 执行用例

```
**注意：暂时只支持工位1
**执行用例时，根据条件找到对应的用例，自动创建一个任务并开始执行，返回任务的id
**此函数为异步,可以调用 GetTaskState 获取执行状态
**deviceNo 为保留参数，默认值为1
请求参数:
//string caseName 用例集名称，支持节点结构，示例：节点1/节点2/用例集名称/用例名称
```

```
//int deviceNo 工位号
返回值：
//任务的Id
异常信息：
1.暂时只支持工位1
2.正在执行测试任务中，无法执行当前动作
3.未找到该用例或该用例未编译
4.其他异常
int ExecCase(string caseName, int deviceNo);
```

4.2 执行用例（按用例Id）

```
**注意：暂时只支持工位1
**执行用例时，根据条件找到对应的用例，自动创建一个任务并开始执行，返回任务的id
**此函数为异步,可以调用 GetTaskState 获取执行状态
**deviceNo 为保留参数，默认值为1
请求参数：
//int testCaseId 用例Id
//int deviceNo 工位号
返回值：
//任务的Id
异常信息：
1.暂时只支持工位1
2.正在执行测试任务中，无法执行当前动作
3.未找到该用例或该用例未编译
4.其他异常
int ExecCaseById(int testCaseId, int deviceNo);
```

4.3 执行任务

```
**注意：暂时只支持工位1
**执行任务时，根据条件找到对应的任务，然后创建一个副本并开始执行，返回任务的id
**此函数为异步，可以调用 GetTaskState 获取执行状态
**deviceNo 为保留参数，默认值为1
请求参数：
//string taskName 任务名称，支持节点结构，示例如：节点1/节点2/任务名称
//int deviceNo 工位号
返回值：
//任务的Id
异常信息：
1.暂时只支持工位1
2.正在执行测试任务中，无法执行当前动作
3.未找到该任务
4.其他异常
int ExecTask(string taskName,int deviceNo);

**执行任务和执行用例的对比：
1.）执行用例，每次只能指定一个用例，执行一次
2.）执行任务，可以提前选择多个用例，设置好循环次数
3.）执行用例和执行任务，都会生成一个新的任务，并返回Id
```

4.4 执行任务（按任务id）

```
**注意：暂时只支持工位1
**执行任务时，根据条件找到对应的任务，然后创建一个副本并开始执行，返回任务的id
**此函数为异步，可以调用 GetTaskState 获取执行状态
**deviceNo 为保留参数，默认值为1
请求参数：
//int taskId 任务Id
//int deviceNo 工位号
返回值：
//任务的Id
异常信息：
1.暂时只支持工位1
2.正在执行测试任务中，无法执行当前动作
3.未找到该任务
4.其他异常
int ExecTaskById(int taskId, int deviceNo);

**执行任务和执行用例的对比：
1.）执行用例，每次只能指定一个用例，执行一次
2.）执行任务，可以提前选择多个用例，设置好循环次数
3.）执行用例和执行任务，都会生成一个新的任务，并返回Id
```

4.5 停止任务

```
 请求参数：
//int taskId 任务的Id
异常信息：
1.其他异常
void StopTask(int taskId);
```

4.6 生成任务报告

```
请求参数：
//int taskId 任务的Id
返回值：
//报告文件名称（只报告名称）
异常信息：
1.正在执行测试任务中，无法执行当前动作
2.任务不存在
3.其他异常
string GenerateTaskReport(int taskId);
```

4.6.1 生成任务报告V2

```
请求参数:
//int taskId 任务的Id
返回值:
//报告文件名称(绝对路径),如产生多份报告将用逗号分割
异常信息:
1.正在执行测试任务中，无法执行当前动作
2.任务不存在
3.其他异常
string GenerateTaskReportV2(int taskId);
```

4.7 获取所有的用例集信息

```
返回值:
//字符串,用json表示用例集相关信息
json格式如下:
[
    {
        "SetName":"用例集1",      //用例集名称
        "SetId":11,               //用例集id
        "SetDesc":"用例集1的描述" //用例集描述
    },
    {
        "SetName":"用例集2",
        "SetId":12,
        "SetDesc":"用例集2的描述"
    }
]
string GetCaseSets();
```

4.8 根据用例集id来获取该用例集下的所有用例

```
请求参数:
//int caseSetId 用例集Id
返回值:
//字符串,用json表示相关用例信息
json格式如下:
[
    {
        "CaseName":"用例1",       //用例名称
        "CaseId":1,               //用例id
        "CaseDesc":"用例1的描述", //用例描述
        "SetId":11                //用例集id
    },
    {
        "CaseName":"用例2",
        "CaseId":2,
        "CaseDesc":"用例2的描述",
        "SetId":12
    }
]
string GetCasesByCaseSetId(int caseSetId);
```

4.9 根据用例id获取用例信息

```
请求参数:
//int caseId 用例id
返回值:
//字符串,用json表示相关用例信息
json格式如下:
{
    "SetId":721,              //用例集id
    "CaseId":1178,            //用例id
    "CaseName":"",            //用例名称
    "CaseDesc":"",            //用例描述
    "ModifyTime":"2023-02-28T15:39:21" //用例最后修改时间
}
异常信息:
1.查找失败:即未找到该用例
//{"ret":false,"error":"用例:xxx不存在"}
string GetCaseByCaseId(int caseId)
```

4.10 根据用例集名称/用例名称获取用例信息

```
请求参数:
//string name 用例集名称/用例名称
返回值:
//字符串,用json表示相关用例信息
json格式如下:
{
    "SetId":721,              //用例集id
    "CaseId":1178,            //用例id
    "CaseName":"",            //用例名称
    "CaseDesc":"",            //用例描述
    "ModifyTime":"2023-02-28T15:39:21" //用例最后修改时间
}
异常信息:
1.查找失败:即输入的参数不能为空
//{"ret":false,"error":"输入的参数为空"}
2.查找失败:输入有误
//{"ret":false,"error":"输入有误,正确格式:用例集名称/用例名称"}
3.查找失败:用例集:xxx不存在
//{"ret":false,"error":"用例集:xxx不存在"}
4.查找失败:用例集xxx的用例xxx不存在
//{"ret":false,"error":"用例集xxx的用例xxx不存在"}
string GetCaseByName(string name)
```

4.11 获取所有用例

```
返回值:
//字符串,用json表示所有用例信息
json格式如下:
[
    {
        "CaseName":"用例1",       //用例名称
        "CaseId":1,               //用例id
        "CaseDesc":"用例1的描述", //用例描述
```

```
        "SetId":11              //用例集id
    },
    {
        "CaseName":"用例2",
        "CaseId":2,
        "CaseDesc":"用例2的描述",
        "SetId":12
    }
]
string GetAllCases();
```

4.12 获取所有用例集以及用例

返回值：

```
//字符串,用json表示所有用例集用例信息
json格式如下:
[
    {
        "SetName":"用例集",
        "SetId":12,
        "SetDesc":"用例集的描述",
        "Cases":[
            {
                "Name":"用例1",
                "Id":1,
                "Desc":"用例1的描述",
                "SetId":12
            }
        ]
    }
]
string GetAllSetAndCase();
```

4.13 创建任务

请求参数：

```
//string task: 创建任务的json字符串
创建任务请求参数json:
{
"TaskName": "任务名称", //任务名称 不能为空
"TaskDesc": "任务描述", //任务描述
"ExePolicy": 0,          //执行策略，0按次数 1 按结束时间
"ExeTimes": 10,          //ExePolicy为0时有效，指定执行次数
"FinishTime": "2022/08/16 17:18:19", // ExePolicy为1时有效 zhi指定截止时间
"TaskMode": 0, // 0 交互模式 1: 无效（暂不使用）2 并行模式 3 指定工位模式
// 执行的用例，不能为空且是已编译过的用例
"ExeCases": [
    {
        "CaseId": 1,    //指定执行的用例id
        "SetId": 11,    //用例所属用例集id
        "ExeTimes": 10, //执行次数
        "FailExecutionPolicy": 0, //失败策略 0 失败后执行跳过(默认) 1 失败后暂停 2 失败后重测
```

```
        "FailRetestCount": 10,     //失败后重测最大次数 在FailExecutionPolicy为2时有效。
        "FailExecutionSubCaseId": -1 //失败后暂停、跳过、重测前指定执行的子用例id，如果不指定则为-1
    }
    ]
},
"ExtraInfo": { //额外信息，为key，value形式，只允许给定的key
        "key": "value"
}
```

请求参数示例：

```
{
    "TaskName": "接口任务",
    "TaskDesc": "接口任务描述",
    "ExePolicy": 0,
    "ExeTimes": 10,
    "FinishTime": "2022/09/29 10:48:19",
    "TaskMode": 0,
    "ExeCases": [{
        "CaseId": 612,
        "SetId": 132,
        "ExeTimes": 10,
        "FailExecutionPolicy": 0,
        "FailRetestCount": 10,
        "FailExecutionSubCaseId": -1
    }]
}
```

返回值：

1.创建成功

```
//{
    "TaskName":"创建",    //任务名称
    "TaskId":3,           //任务ID
    "ret":true,           //返回结果
    "error":""            //错误信息
  }
```

2.创建失败：创建任务参数不能为空，即输入的task不能为空

```
//{"TaskName":null,"TaskId":0,"ret":false,"error":"创建任务参数不能为空"}
```

3.创建失败：未选择要执行的用例

```
//{"TaskName":null,"TaskId":0,"ret":false,"error":"未选择要执行的用例"}
```

4.创建失败：任务的名称不能为空

```
//{"TaskName":null,"TaskId":0,"ret":false,"error":"创建任务的名称不能为空"}
```

5.创建失败：任务名称有同名情况

```
//{"TaskName":null,"TaskId":0,"ret":false,"error":"任务: {param.TaskName} 已存在"}
```

6.创建失败：用例ID不存在存在

```
//{"TaskName":null,"TaskId":0,"ret":false,"error":"用例: {exeCase.CaseId}不存在"}
```

7.创建失败：该用例未编译

```
//{"TaskName":null,"TaskId":0,"ret":false,"error":"用例: {exeCase.CaseId}未编译"}
```

8.创建失败：内部抛异常

```
//{"TaskName":null,"TaskId":0,"ret":false,"error":"创建任务失败，内部错误，错误信息: {e.Message}"}
string CreateTask(string task);
```

4.14 更新任务

请求参数：

```
//int taskId: 任务Id
//string task: 更新任务的json字符串
```

更新任务请求参数json：

```
{
"TaskName": "任务名称", //任务名称 不能为空
"TaskDesc": "任务描述", //任务描述
"ExePolicy": 0,          //执行策略，0按次数 1 按结束时间
"ExeTimes": 10,          //ExePolicy为0时有效，指定执行次数
"FinishTime": "2022/08/16 17:18:19", // ExePolicy为1时有效 zhi指定截止时间
"TaskMode": 0, // 0 交互模式 1: 无效（暂不使用）2 并行模式 3 指定工位模式
// 执行的用例，不能为空且是已编译过的用例
"ExeCases": [
    {
        "CaseId": 1,     //指定执行的用例id
        "SetId": 11,     //用例所属用例集id
        "ExeTimes": 10, //执行次数
        "FailExecutionPolicy": 0, //失败策略 0 失败后执行跳过(默认) 1 失败后暂停 2 失败后重测
        "FailRetestCount": 10,     //失败后重测最大次数 在FailExecutionPolicy为2时有效。
        "FailExecutionSubCaseId": -1 //失败后暂停、跳过、重测前指定执行的子用例id，如果不指定则为-1
    }
    ]
},
"ExtraInfo": { //额外信息，为key，value形式，只允许给定的key
        "key": "value"
}
```

请求参数示例：

```
{
    "TaskName": "接口任务",
    "TaskDesc": "接口任务描述",
    "ExePolicy": 0,
    "ExeTimes": 10,
    "FinishTime": "2022/09/29 10:48:19",
    "TaskMode": 0,
    "ExeCases": [{
        "CaseId": 612,
        "SetId": 132,
        "ExeTimes": 10,
        "FailExecutionPolicy": 0,
        "FailRetestCount": 10,
        "FailExecutionSubCaseId": -1
    }]
}
```

返回值：

```
{
    ret:true,
    error:"errormsg"
}
string UpdateTask(int taskId,string task)
```

4.15 获取失败的TestExecution\查询指定任务下失败的用例

```
请求参数：
//int taskId 任务的Id
返回值：
解释：FailedExecutionIds为int数组，返回的日志所属的Execution Id集合
1.查询成功
//{"FailedExecutionIds":[2284,2285,2286,2287,2288],"ret":true,"error":null}
2.未查询到执行日志
//{"FailedExecutionIds":null,"ret":false,"error":"未查询到执行日志"}
string GetFailedExectutions(int taskId);
```

4.16 获取当前客户端执行状态

```
返回值：
//true:执行状态 false:空闲状态
bool GetTaskExecuteState();
```

4.17 获取硬件异常信息

```
**注意： 当API执行失败时， 调用该函数获取硬件错误状态,硬件问题导致的异常,需要重启设备及软件来恢复
//硬件状态码解释：
//0：正常状态
//1XXX：机械臂异常
//2XXX：相机异常
//3XXX：外设异常，如CAN、USB、IO板卡、光栅等
//4XXX：未知异常
返回值：
{
    "ret":true,         //执行结果 true or fasle, 表示执行是否成功, 异常信息由状态码来判
断.
    "error":"",         //捕获的异常信息
    "ErrCode":"",       //硬件状态码
    "RobotSN":"" ,      //机械臂IP地址
    "RobotError":"" ,   //机械臂返回的异常信息
    "CamSN":""          //相机的SN号
    "CamError":"" ,     //相机返回的异常信息
}
string GetHardwareError();
```

4.18 复测指定的任务

```
请求参数:
//string retestTaskStr: 复测任务json字符串
请求参数示例:
{
   "TaskId": 1,
   "ExecutionIds": [1, 4, 7]
}
返回值：json字符串
{
    "ret": true, // true 表示执行成功
    "error": "" // ret 为false时此处为错误信息
}
string RetestTask(string retestTaskStr)
```

4.19 暂停任务

```
请求参数:
//int taskId: 待暂停任务id
返回值：json字符串
{
    "ret": true, // true 表示执行成功
    "error": "" // ret 为false时此处为错误信息
}
string PauseTask(int taskId)
```

4.20 继续任务

```
请求参数:
//int taskId: 上次暂停的任务id
返回值：json字符串
{
    "ret": true, // true 表示执行成功
    "error": "" // ret 为false时此处为错误信息
}
string ContinueTask(int taskId)
```

4.21 根据任务id删除任务

```
请求参数:
//int taskId: 已终止的任务id
返回值：json字符串
1.如果任务存在
{
    "ret": true, // true 表示执行成功
    "error": $"删除任务:{taskId}成功" // ret 为false时此处为错误信息
}
2.如果任务不存在
{
    "ret": false, // true 表示执行成功
    "error": $"任务:{taskId}不存在" // ret 为false时此处为错误信息
}
string DeleteTaskById(int taskId)
```

4.22 根据任务名称删除任务

请求参数：

```
//string taskName: 任务名称
1.如果任务存在
{
    "ret": true, // true 表示执行成功
    "error": $"删除任务:{taskName}成功" // ret 为false时此处为错误信息
}
2.如果任务不存在
{
    "ret": false, // true 表示执行成功
    "error": $"任务:{taskName}不存在" // ret 为false时此处为错误信息
}
string DeleteTaskByName(string taskName)
```

4.23 根据任务名称获取任务

请求参数：

```
//int taskName: 任务名称
返回值：
//任务信息json字符串，如下：
{
    "ExePolicy": 0, // 执行策略 0 按次数 1 按结束时间
    "IsExePolicyRandom": false, // 忽略，冗余字段
    "IsExePolicyByTimes": true, // 忽略
    "isExePolicyByFinishTime": false, // 忽略
    "ExeTimes": 10, // 执行次数，ExePolicy=0时有效
    "FinishTime": "0001-01-01T00:00:00", // 结束时间，ExePolicy=1时有效
    "CreateTime": "2022-09-28T14:58:59.321", //任务的创建时间
    "EmgStop": false, // 忽略
    "FailureTolerance": 0, // 容忍失败次数
    "CurrExeId": 0, // 当前执行Id
    "CurrExeTimes": 0, // 当前用例执行次数
    "CurrTaskTimes": 0, // 当前任务执行次数
    "LastRunTime": "", // 最后一次执行时间
    "TaskMode": 0, //任务模式 0 多机交互 固定值
    "Executions": Array[1], // 执行列表
    "ExecutionLogs": [ // 执行日志列表

    ],
    "TaskExcuter": null, // 忽略
    "Id": 3, // 任务id
    "Name": "任务名称", // 任务名称
    "Desc": "任务描述", // 任务描述
    "PropertyArgs": {
        "keys": [ // 任务扩展字段key
            "deviceList",
            "taskMode",
            "serverID",
            "LastRunTime"
        ],
        "dict": {
            //deviceList、taskMode、serverID、LastRunTime同类型
```

```
            "deviceList": { // 任务扩展字段对应的信息
                "Name": "deviceList", // 扩展字段名称
                "Value": "1", // 扩展字段值
                "Tip": "", // 扩展字段提示
                "Key": "deviceList", // 扩展字段key
                "Visible": true, // 忽略
                "TypeString": null // 忽略
            },
            "taskMode": {},
            "serverID": {},
            "LastRunTime": {}
        },
        "PreArgKeys": [
            "deviceList",
            "taskMode",
            "serverID",
            "LastRunTime"
        ]
    }
}
string GetTaskByName(string taskName)
```

4.24 根据任务id获取任务

```
请求参数:
//int taskId: 任务id
返回值:
//同4.22
string GetTaskById(int taskId)
```

4.25 判断任务名称是否可用

```
请求参数:
//string taskName: 任务名称
返回值:
//true 可用, false 不可用
bool CheckTaskName((string taskName))
```

4.26 根据任务id获取任务信息详情

```
请求参数:
//int taskId: 任务id
返回值:
{
"ExePolicy": 0,
"IsExePolicyRandom": false,
"IsExePolicyByTimes": true,
"isExePolicyByFinishTime": false,
"ExeTimes": 1,
"FinishTime": "2023-02-28T15:39:57",
"CreateTime": "2023-02-28T15:40:14",
"EmgStop": false,
```

```
"FailureTolerance": 0,
"CurrExeId": 0,
"CurrExeTimes": 0,
"CurrTaskTimes": 0,
"LastRunTime": "2023-02-28 15:40:16.757",
"TaskMode": 0,
"Executions": [
    {
        "TaskId": 148,
        "CaseId": 1179,
        "CaseType": 0,
        "SetId": 721,
        "ExeTimes": 1,
        "BindDevice": "0",
        "ExeTimesStr": "1",
        "ExeOrder": 0,
        "FailExecutionPolicy": 0,
        "FailExecutionSubCaseId": -1,
        "FailLogCount": 0,
        "FailRetestCount": 1,
        "FailExecutionPolicyStr": "默认",
        "Id": 2461,
        "Name": "测试点击、滑动",
        "Desc": "",
        "PropertyArgs": {
            "keys": [
                "defeattype",
                "defeatcount",
                "subcaseid",
                "failPolicySync"
            ],
            "dict": {
                //defeattype、defeatcount、subcaseid、failPolicySync同类型
                "defeattype": {
                    "Name": "defeattype",
                    "Value": "0",
                    "Tip": "0",
                    "Key": "defeattype",
                    "Visible": true,
                    "TypeString": null
                },
                "defeatcount": {},
                "subcaseid": {},
                "failPolicySync": {}
            },
            "PreArgKeys": [
                "defeattype",
                "defeatcount",
                "subcaseid",
                "failPolicySync"
            ]
        }
    }
],
"ExecutionLogs": [
```

```
    {
        "ExecutionId": 2461,
        "StartTime": "2023-02-28T15:40:16.766",
        "FinishTime": "2023-02-28T15:40:18.674",
        "ExeResult": true,
        "ExeResultNew": 1,
        "IsRetest": false,
        "CaseId": 1179,
        "SetId": 721,
        "TaskId": 148,
        "LogsDir": "",
        "Loop": 1,
        "Times": 0,
        "WorkFolder": "D:\\工作\\GR\\GR主项目
\\src\\Release\\logs\\Robot\\taskLogs\\148\\1179\\20230228154016766",
        "DeviceNo": 1,
        "CaseType": 0,
        "Id": 36815,
        "Name": "测试点击、滑动",
        "Desc": null,
        "PropertyArgs": {
            "keys":
["currentMax","currentMin",".....","flickerType","customLogs","下面dict对象的属性"]
            "dict": {
                //dict中属性的类型均与currentMax一致
                "currentMax": {
                    "Name": "currentMax",
                    "Value": "0",
                    "Tip": "0",
                    "Key": "currentMax",
                    "Visible": true,
                    "TypeString": null
                },
                "currentMin": {},
                "currentAvg": {},
                "current_Low": {},
                "voltage": {},
                "power": {},
                "current_th": {},
                "current_type": {},
                "current_start": {},
                "current_end": {},
                "screen_rectangle": {},
                "imageFolder": {},
                "start": {},
                "finish": {},
                "marks": {},
                "avg": {},
                "allowValue": {},
                "frameRate": {},
                "caseMark_Dev_lcd": {},
                "fluency_startIndex": {},
                "fluency_finishIndex": {},
                "fluency_rectangle": {},
                "fluency_caton_checktype": {},
```

```
            "fluency_caton_time_th": {},
            "fluency_gray_threshold": {},
            "fluency_caton_threshold": {},
            "fluency_device_frameRate": {},
            "fluency_test_range": {},
            "fluency_caton_totail": {},
            "fluency_caton_max_combo": {},
            "fluency_catonNumber": {},
            "fluency_caton_percent": {},
            "fluency_caton_resum": {},
            "fluency_image_folder": {},
            "fluency_framerate_threshold": {},
            "fluency_framerate_avg": {},
            "fluency_caton_total_time": {},
            "fluency_meet_conditions": {},
            "fluency_caton_value_th": {},
            "fluency_caton_count_th": {},
            "fluency_key_event_type": {},
            "fluency_key_event_name": {},
            "fluency_ratio": {},
            "resultDetail": {},
            "fluency_caton_max_combo_threshold": {},
            "fluency_catonNumber_threshold": {},
            "followHandArgs": {},
            "devicepercamprovider": {},
            "startIndex": {},
            "finishIndex": {},
            "smooth_rectangle": {},
            "smooth_match_threshold": {},
            "smooth_border_threshold": {},
            "smooth_image_folder": {},
            "smooth_device_frame_rate": {},
            "smooth_camera_frame_rate": {},
            "smooth_detect_direction": {},
            "smooth_checktype": {},
            "smooth_device_lcdrect": {},
            "Thermalproductid": {},
            "rectangle": {},
            "caseType": {},
            "threshold": {},
            "deviceNo": {},
            "min": {},
            "max": {},
            "maxContinuousLost": {},
            "lost": {},
            "lostRatio": {},
            "pause": {},
            "ratio": {},
            "retestresult": {},
            "retestcount": {},
            "brightThreshold": {},
            "flickerType": {},
            "customLogs": {}
        },
```

```
            "PreArgKeys":
["currentMax","currentMin",".....","flickerType","customLogs","上面dict对象的属性"]
        }
    }
],
"TaskExcuter": null,
"Id": 148,
"Name": "测试点击滑动坐标",
"Desc": "",
"PropertyArgs": {
    "keys": [
        "deviceList",
        "taskMode",
        "serverID",
        "LastRunTime"
    ],
    "dict": {
        //deviceList、taskMode、serverID、LastRunTime对象类型一样
        "deviceList": {
            "Name": "deviceList",
            "Value": "0",
            "Tip": "",
            "Key": "deviceList",
            "Visible": true,
            "TypeString": null
        },
        "taskMode": {},
        "serverID": {},
        "LastRunTime": {}
    },
    "PreArgKeys": [
        "deviceList",
        "taskMode",
        "serverID",
        "LastRunTime"
    ]
}
}
string GetTaskDetail(int taskId)
```

4.27 获取当前机械手状态

返回值：

```
//json字符串
{
    "ret": true, // true 表示当前机械手可用
    "error": "" // ret 为false时此处为错误信息
}
string GetRobotStatus()
```

4.28 获取所有节点信息

返回值：

```
//json字符串
[
  {"Id":3,"Name":"A","ParentId":0},
  {"Id":4,"Name":"B","ParentId":0},
  {"Id":5,"Name":"C","ParentId":4}
]
string GetAllNodes()
```

4.29 根据节点id获取用例集列表

请求参数：

```
//int nodeId 节点id
```

返回值：返回用例集列表

```
//json字符串
[{"SetId":32,"SetName":"App展开","SetDesc":""}]
string GetCaseSetsByNodeId(int nodeId)
```

4.30 通过任务ID获取任务的错误、异常执行结果

参考异常错误分类：

1：相机异常、断连
2：机械手运动异常、断连
3：双指安全距离碰撞风险预警
4：光栅、压力急停信号异常
5：用例执行异常 例：图像识别失败、硬件串口连接失败等
6：ADB检测异常报点

请求参数：

```
//int taskId 任务id
```

返回值：返回用例集列表

```
//json字符串
```

返回值：

```
    [{
    "TaskId": 36,
    "SetId": 27,
    "CaseId": 72,
    "CaseName": "测试1",
    "StartTime": "2023-08-14T13:19:29.07",
    "FinishTime": "2023-08-14T13:19:30.999",
    "ExeResult": "Error",
    "ErrorLogs": ["ERROR:主手长按副手点击 出现异常：主手的z轴行程异常，必须要大于0"]
    }]
string GetErrorTaskResultById(int taskId)
```

4.31 获取全部任务的错误或者异常的执行结果

参考异常错误分类：

1：相机异常、断连
2：机械手运动异常、断连
3：双指安全距离碰撞风险预警
4：光栅、压力急停信号异常

```
5：用例执行异常 例：图像识别失败、硬件串口连接失败等
6：ADB检测异常报点
返回值：返回用例集列表
//json字符串
 返回值：
    [{
    "TaskId": 36,
    "SetId": 27,
    "CaseId": 72,
    "CaseName": "测试1",
    "StartTime": "2023-08-14T13:19:29.07",
    "FinishTime": "2023-08-14T13:19:30.999",
    "ExeResult": "Error",
    "ErrorLogs": ["ERROR:主手长按副手点击 出现异常：主手的z轴行程异常，必须要大于0"]
    }]
string GetErrorTaskResult()
```

4.32 获取上次外部命令返回结果

```
返回值：
    {
     "ret":true,
     "error":"" //如果出错，有错误信息 ret:false
    }
string GetLastprocessReturns()
```

4.33 导出用例

```
请求参数：
//string fileName 完整文件路径包含文件名和后缀 .ZIP，比如 D:\\Export\\export.zip
//string caseIds 用例ID  多个逗号隔开,-1导出所有用例
//string caseSetIds 用例集ID 多个逗号隔开,-1导出所有用例集
//string importDevice 是否导出设备信息
返回值：
    {
     "ret":true,
     "error":"" //如果出错，有错误信息 ret:false
    }
说明：
//用例id和用例集id用户可以选填，但两者不能同时为空字符串，也可以同时都填，程序会过滤重复部分
string ExportCases(string fileName, string caseIds, string caseSetIds, bool
importDevice)
```

4.34 导入用例

```
请求参数:
//string fileName 完整文件路径包含文件名和后缀 .ZIP，比如 D:\\Export\\export.zip
//bool overWrite 当有重复用例时，是否覆盖
返回值:
    {
     "ret":true,
     "error":"" //如果出错，有错误信息 ret:false
    }
string ImportCase(string fileName, bool overWrite)
```

4.35 编译用例

```
请求参数:
//string caseSets 用例集ID  多个逗号隔开,-1编译所有用例集
返回值:
    {
     "ret":true,
     "error":"" //如果出错，有错误信息 ret:false
    }
string Compile(string caseSets)
```

5.PCB相关

```
使用场景:程控电源开关、usb切换
```

5.1 程控电源，控制指定通道的电源通断(兼容操作空调的功能)

```
请求参数:
//int channelNo 通道号 1~n，此是在我们配置的多个串口里面的排序，从1开始
//int onOffStatus 开关状态 0：关  1：开
void ChangePowerStatu(int channelNo, int onOffStatus)
```

5.2 USB切换

```
请求参数:
//i32 channelNo 通道号 1~n，此是在我们配置的多个串口里面的排序，从1开始
//string type 控制类型，关键词为：快充、浮地、接地、OTG、接通USB、断开USB
void USBChargeControl(int channelNo, string type)
```